Social & Media Platform APIs

Week 12 · APIs module · Textbook Chapter 12

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

November 2, 4 & 6, 2026

This week at a glance

Last week the data came from governments obliged to publish it. This week it comes from platforms that share it *at their discretion*, and can stop. We read what federal agencies post, every class live coding, from one lab notebook in three parts.

Monday | Lab, part 1: Bluesky

- An open API with no key: who posts, and is it the agency?

Wednesday | Lab, part 2: more of Bluesky

- More posts than one request returns, and an agency's rhythm

Friday | Lab, part 3: YouTube; then your project

- A keyed API for the same agencies' videos; 15 minutes on your portfolio checkpoint

One key, for Friday

`YOUTUBE_KEY`, a YouTube Data API key from a **personal** Google account. Set it as an environment variable, close the terminal, open a new one, and start Jupyter from there.

Due Sunday, Nov 8

Chapter 12's **Recommended Exercises**, from its companion notebook, and portfolio checkpoint 2, *Collect and clean your data*. Graded on good-faith attempt to complete rather than getting everything working.

1. Monday · Live-Coding Lab

Daily note questions

- **Someone reviewing API keys? —** Not really, but it acts like an account+password so they can (and do!) track how much a key is being used
- **Where to store API keys? —** in a text file and copy-paste into `getpass` (easy), in a JSON file you read into notebook (medium), or store as an environment variable (medium-hard)

This week, every class is live coding

Find it | `labs/` in the course repository

1. Open the **labs** folder of `cuinfoscience/INF04617-Fall2026` on GitHub
2. Open `week-12`, click `lab-12.ipynb`, then **Download raw file**
3. Open it in JupyterLab in the `webdata` environment

Use it | one part a day

- I write the code on screen, and you write it in your copy
- Put **your** email in the User-Agent in Step 0
- Today: Part 1, Steps 0–3; no key, no account
- Wednesday and Friday: download the new copy and start at that part's setup step

Solutions

`lab-12-key.ipynb`, beside it: every cell filled in and run. Fall behind? Copy the cell from the key and keep going.

Nothing to submit

The lab is practice. After each step, a **Discuss** box holds questions we'll talk over before the next.

Error flavors

A `KeyError` on a field means the answer wasn't what you expected: print it and look. A request that fails for everyone is *broken*: tell me, or open an issue.

Platforms share at their discretion

Government APIs exist because a law says the data is public. A platform's API exists because the company chooses to offer it, and it can choose again:

- **Reddit** closed self-service API access on November 11, 2025: a new app now needs Reddit's approval.
- **Spotify** has required every new developer app's owner to have Premium since February 11, 2026, and removed top tracks, popularity, and follower counts.
- **Bluesky** is built on an open protocol: its public API answers anyone, with no account or key.

Chapter 12 tells the Reddit and Spotify stories; the lab reads Bluesky, then YouTube.

Social data is people data

A person made almost every post. Read the terms of service; collect the minimum you need; don't quote a post if a search could find its author. This week we read **federal agencies**, and keep private people's posts and handles out of every output.

Start from the page a person reads



The National Weather Service on bsky.app, captured 2026-11-02 (chapter 12, figure 12.1).

Each marker is a field

1. The handle, a domain: `handle`
2. The blue check: `verification`
3. The counts: `followersCount`, `postsCount`
4. The posts, newest first: `getAuthorFeed`

The page rounds to 63K followers and 1.3K posts. The API gives the exact counts.

From the website to the API: one agency's profile

The National Weather Service at bsky.app/profile/nws.noaa.gov, and the same profile from the API:

```
bsky_base = "https://public.api.bsky.app/xrpc"
profile = requests.get(
    f"{bsky_base}/app.bsky.actor.getProfile",
    params={"actor": "nws.noaa.gov"},
    headers=HEADERS).json()
print(profile["handle"], "|", profile["displayName"])
print("followers:", profile["followersCount"],
      "| posts:", profile["postsCount"])
# nws.noaa.gov | National Weather Service
# followers: 63020 | posts: 1394
```

The method's name is part of the address. No key, no login: Bluesky's **AppView** answers reads for anyone.

Docs: `getProfile`, which lists every field; every method: the HTTP API reference

Is it really the agency?

Two proofs, both in the API's answer:

- The **handle is a domain**, `nws.noaa.gov`: only whoever controls the domain can take it
- A **blue check**: `"verification"` says Bluesky itself verified it

A search for "National Park Service" also finds "Alt National Park Service", with neither. Collect its posts as the agency's, and your study is wrong.

Five agencies in one table

```
agencies = ["noaa.gov", "nws.noaa.gov",
            "climate.noaa.gov", "noaacomms.noaa.gov",
            "federalreserve.gov"]
rows = []
for handle in agencies:
    p = requests.get(
        f"{bsky_base}/app.bsky.actor.getProfile",
        params={"actor": handle}, headers=HEADERS).json()
    rows.append({"handle": p["handle"],
                 "followers": p["followersCount"],
                 "posts": p["postsCount"],
                 "joined": p["createdAt"][:10]})
accounts = pd.DataFrame(rows).sort_values(
    "followers", ascending=False)
```

Docs: `getProfile`; `getProfiles` takes up to 25 accounts in one request.

Handle	Followers	Posts	Joined
nws.noaa.gov	63,020	1,394	2025-01
climate.noaa.gov	46,966	93	2025-01
noaa.gov	41,937	150	2025-01
federalreserve.gov	8,405	1,029	2024-12
noaacomms.noaa.gov	5,620	117	2025-01

These are the federal accounts with a `.gov` handle on November 2; four are NOAA's. NASA and the Census Bureau have none.

Microsprint

Not every official account has a domain: `librarycongress.bsky.social` and `stlouisfed.bsky.social` (FRED's home) are both verified instead.

What an agency posted this week

```
params = {"actor": "nws.noaa.gov", "limit": 25,
          "filter": "posts_no_replies"}
feed = requests.get(
    f"{bsky_base}/app.bsky.feed.getAuthorFeed",
    params=params, headers=HEADERS).json()["feed"]
posts = []
for item in feed:
    if "reason" in item: # a repost
        continue
    post = item["post"]
    posts.append({"time": post["indexedAt"][:16],
                  "likes": post["likeCount"],
                  "reposts": post["repostCount"],
                  "text": post["record"]["text"][:70]})
```

Docs: `getAuthorFeed`, with the values `filter` takes.

Most reposted, NWS

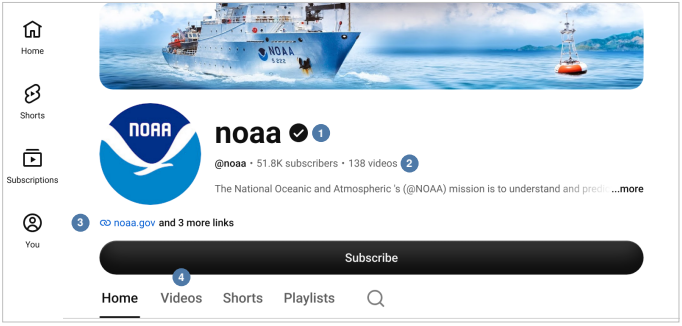
Reposts	Post
80	Daylight Saving Time ends …
33	Updated Extreme Rainfall …
33	Here's a more detailed look …

Why skip reposts?

A repost carries someone else's post, and they may be a private person. Skipping them keeps every handle in our table an agency's.

The answer ends with a `cursor`. Wednesday: what is it for?

Friday's page: an agency's YouTube channel



NOAA's channel on youtube.com, captured 2026-11-02 (chapter 12, figure 12.2).

The same four questions

1. The name: `snippet.title`
2. The handle and counts: `subscriberCount`, `videoCount`
3. A link to `noaa.gov`
4. The videos: the **uploads** playlist, via `playlistItems.list`

Marker 3 proves nothing alone: anyone can link to any site. The proof runs the other way, from the agency's website to the channel.

Unlike Bluesky, every one of these requests needs a key.

Before Friday: your YouTube key

`console.cloud.google.com/apis/library/youtube.googleapis.com`

1. Open the link above, signed in with a **personal** Google account (make a project if it asks), and click **Enable**
2. On the **Credentials** tab: **Create credentials**, then **API key**, restricted to **YouTube Data API v3**
3. Store it with the `webdata` environment, in a terminal (on Windows, the Anaconda Prompt):

```
conda env config vars set YOUTUBE_KEY="your-key" -n webdata
```

Then close that terminal, open a new one, run `conda activate webdata`, and start Jupyter from it.

Screenshots of every step: Friday's first frames, and the lab's "Before Step 8".

It's a password

Never paste the key into a notebook cell or a code file. If it leaks, delete it in the console and make a new one.

The quota

10,000 units a day; most reads cost 1 unit, and searches are capped at 100 a day (Google's guide). Friday's code reads channels and videos, not searches.

2. Wednesday · Live-Coding Lab

Today: Part 2, more of Bluesky

One request returns at most 100 posts. Today:

- Follow the `cursor` for more, as week 10 followed Wikipedia's `continue`
- An agency's rhythm: when it posts, in its own time zone
- Which posts travel, and what a table like that can't tell you

Use it | Steps 4–7

- Download the new `lab-12.ipynb`; start at **Step 4**
- Still no key, no account

Friday needs a key

`YOUTUBE_KEY`, from a personal Google account: Monday's "Before Friday" frame has the steps. Make it today.

More than one page: the `cursor`

```
params = {"actor": "nws.noaa.gov", "limit": 100,
          "filter": "posts_no_replies"}
first = requests.get(
    f"{bsky_base}/app.bsky.feed.getAuthorFeed",
    params=params, headers=HEADERS).json()
# 100 items, from 2026-10-31 back to 2026-10-02

params["cursor"] = first["cursor"]
second = requests.get(
    f"{bsky_base}/app.bsky.feed.getAuthorFeed",
    params=params, headers=HEADERS).json()
# 100 items, from 2026-10-01 back to 2026-08-26
```

Then a given loop: up to five pages, skipping reposts. It stopped at 495 posts, back to April 20, with a cursor still left.

Docs: `getAuthorFeed`'s `cursor` and `limit`

Three ways to page

API	You send back
Wikipedia (wk 10)	`continue`
EIA (wk 11)	an `offset` you count
Bluesky	the `cursor`

When to stop

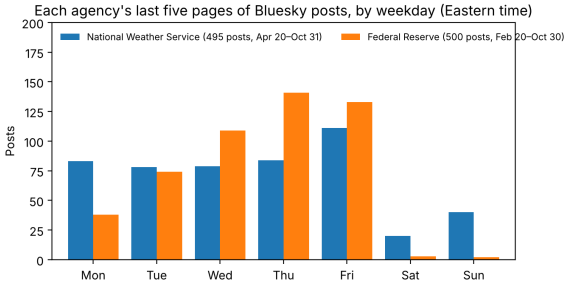
At the end (no cursor), at a count, or at a date. Choose one, and say which when you write it up.

When an agency posts

```
local = nws["created"].dt.tz_convert(
    "America/New_York")
by_day = (local.dt.day_name()
          .value_counts().reindex(days))
by_day.plot.bar(rot=0)
```

Times arrive in UTC: convert to the agency's zone before counting by day or hour.

The weather doesn't take weekends off; the agencies' staff mostly do. The Weather Service peaks on Friday, before the weekend's weather; the Fed posts Wednesday to Friday, and almost never on weekends.



Drawn from the lab's Step 6 and its microsprint, run 2026-11-02 against Bluesky's public API.

What travels

```
nws.groupby("embed")["reposts"].agg(
    ["count", "median"]).sort_values(
    "count", ascending=False)
```

Embed	Posts	Median reposts
images	412	19
video	59	23
external (link card)	13	14
record with media	5	23
record (a quoted post)	3	24
none	3	28

Docs: Bluesky's Posts in-depth, on each kind of embed.

Describe, don't explain

412 of 495 posts carry an image, so the other rows compare a handful of posts. And a post's reach depends on the weather it warns about far more than on its format.

Discuss

What would you need, beyond this table, to say a format *causes* reposts? We left out the agency's replies: what could they tell you, and what would collecting them ask of you?

3. Friday · Live-Coding Lab

Today: Part 3, YouTube's Data API

Lab | Steps 8–11

- A keyed API: check `YOUTUBE_KEY` before you read it
- An agency's channel by its handle, and its newest videos' statistics
- What it cost: every request counts against a daily quota
- Download the new `lab-12.ipynb`; start at **Step 8**

Then | your project, 15 minutes

- Portfolio checkpoint 2, *Collect and clean your data*

`MISSING?`

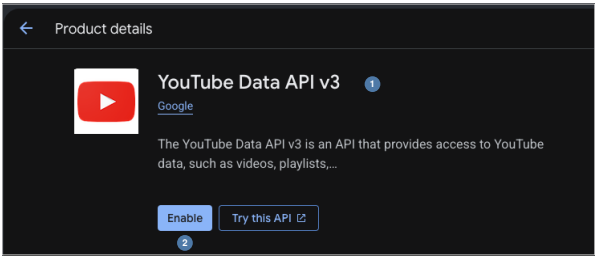
Quit Jupyter, open a new terminal, start it again. Can't restart? Step 8's given cell asks with `getpass()`: paste into the box, press Enter.

No key yet?

The next five frames make one, step by step; the lab's "Before Step 8" has the same steps. Or follow along from the key, and make one after class.

Your YouTube key: start from this link

`console.cloud.google.com/apis/library/youtube.googleapis.com`

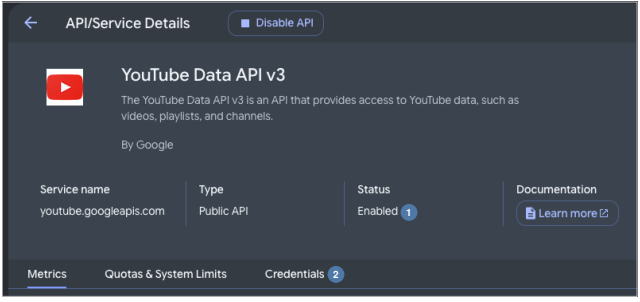


The Google Cloud console, Firefox, 2026-11-06 (chapter 12, figure 12.3).

1. Sign in with a **personal** Google account. Asked to select a project? **New Project**, name it `info4617`, **Create**.
2. Check the page says **YouTube Data API v3** 1, and click **Enable** 2.

The console is built for companies that run many services; this link skips its menus.

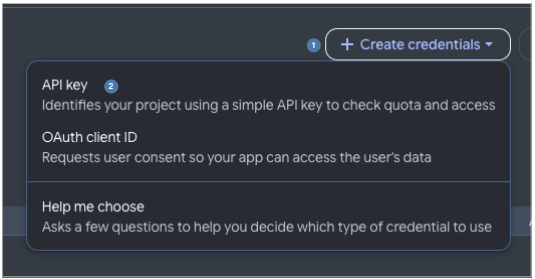
Your YouTube key: open Credentials



The Google Cloud console, Firefox, 2026-11-06 (chapter 12, figure 12.4).

3. The page becomes **API/Service Details**: Status, **Enabled** 1. Click the **Credentials** tab 2.

Your YouTube key: create an API key

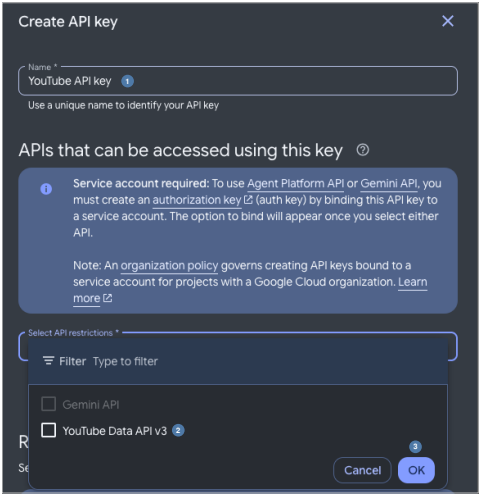


The Google Cloud console, Firefox, 2026-11-06 (chapter 12, figure 12.5).

4. Click **Create credentials** 1, and choose **API key** 2.

Not **OAuth client ID**: that is for a program acting on a person's own YouTube account. Reading public channels needs only a key.

Your YouTube key: restrict it to YouTube



The top of the dialog, Firefox, 2026-11-06; chapter 12, figure 12.6, shows all of it.

5. In **Create API key**:
   - Name the key 1
   - Skip the blue **Service account required** note: it's about Gemini
   - Open **Select API restrictions**, tick **YouTube Data API v3** 2, click **OK** 3
   - Below this: leave **Application restrictions** at **None**, since your laptop's IP address changes from network to network
   - Click **Create**

Your YouTube key: copy it, and store it

6. **Copy the key**, and paste it only into the next step
7. Store it with the `webdata` environment, in a terminal (on Windows, the Anaconda Prompt):

```
conda env config vars set YOUTUBE_KEY="your-key" -n webdata
```

Then **quit Jupyter, close that terminal, open a new one**, run `conda activate webdata`, and start Jupyter from it. A terminal reads its variables when it opens.

It's a password

Never paste the key into a notebook cell, a code file, or a message. If it leaks, delete it on the Credentials page and make a new one.

Week 11's other safe ways work too: a JSON file kept out of git, or `getpass()`, which Step 8's given cell uses.

A channel by its handle

```
yt_base = "https://www.googleapis.com/youtube/v3"
params = {"part": "snippet,statistics,contentDetails",
          "forHandle": "@NOAA", "key": youtube_key}
channel = requests.get(f"{yt_base}/channels",
    params=params, headers=HEADERS).json()["items"][0]
print(channel["snippet"]["title"],
      channel["statistics"]["subscriberCount"],
      channel["statistics"]["videoCount"])
uploads = channel["contentDetails"][
    "relatedPlaylists"]["uploads"]
```

`part` names the groups of fields you want back; `uploads` is the playlist of every video the channel has posted.

Docs: `channels.list`; the channel resource, field by field

Is it the agency's?

Check that the agency's own website links to the channel. `@NOAA`, `@NOAAClimate`, `@federalreserve`, `@stlouisfed`, `@uscensusbureau`, and `@NASA` all have channels.

Rounded

YouTube rounds `subscriberCount` down to three significant figures, so a small change won't show.

A channel's newest videos, and what they cost

```
params = {"part": "contentDetails",
          "playlistId": uploads, "maxResults": 50,
          "key": youtube_key}
items = requests.get(f"{yt_base}/playlistItems",
    params=params, headers=HEADERS).json()["items"]
video_ids = []
for item in items:
    video_ids.append(item["contentDetails"]["videoId"])
params = {"part": "snippet,statistics",
          "id": ",".join(video_ids), "key": youtube_key}
```

Then `videos.list`: views and likes for all 50, as strings; `int()` them, and `.get()` a `likeCount` an owner hid.

Docs: `playlistItems.list`, `videos.list`, quota costs

The quota

10,000 units a day; each request here costs 1. Two requests read 50 videos. Searches are capped separately, at 100 a day.

Discuss

How many units would a year of NOAA's uploads cost? We set a hidden `likeCount` to 0: when is that wrong, and how would you count how often it happened?

Friday's last 15 minutes: portfolio checkpoint 2

Due Sunday | Collect and clean your data

- **Collection code** that runs, makes polite requests (a User-Agent that identifies you, pauses between requests), and saves the raw data
- **A cleaning step**, with its code and output, that produces a tidy dataset
- **A small sample** of the cleaned data in your repository, with a data dictionary
- **The README's Data section**: the source, when you collected it, how many records, and what you kept out of the repository and why

Use the 15 minutes

Run your collection code once, here, where you can ask. A platform this week? Its terms of service decide what goes in your repository.

Graded

Two criteria, one point each, on what's pushed by Sunday night.

Key takeaways

1. A platform's API exists **at the platform's discretion**: Reddit and Spotify have closed much of theirs since 2024. Date what you collect, and say what access you had.
2. **Check who is posting** before you collect: a domain handle or a verified check, not a familiar name.
3. **Social data is people data**: read public institutions' accounts, and keep private people's posts and handles out of what you save and show.
4. **Page deliberately**: send back the `cursor` or token you're given, and stop at the end, a count, or a date you chose.
5. **An open protocol** answers anyone; a **keyed API** counts what you use. Both still need a User-Agent.